

09 Neural Networks

Markéta Tranberg, mlap@viauc.dk

April 2026

Table of Contents

- 1 Deep learning
- 2 Architecture of NN
- 3 Forward propagation
- 4 Activation function
- 5 Back propagation
- 6 How to structure a NN
- 7 Code example

Neural networks are not a new thing

- 1943: Computational model for NN, threshold logic (W. McCulloch)
- 1958: Perceptron, the first "modern" neuron, adjusting weights (F. Rosenblatt)
- 1986: Backpropagation, deep NN passing error signals backwards (G. Hinton, D. Rumelhart, R. Williams)
- 1989: System reading hand-written ZIP codes (Y. LeCun)
- 2012: ImageNet competition won by CNN (G. Hinton & team)
- 2018: Turing Award for deep NN (Y. Bengio, G. Hinton, Y. LeCun)

Why NNs are relevant today

- Big data
- GPU power
- Techniques based on NN (CNN, RNN, transformers, GAN)
convolutional generative adversarial network
recurrent

Applications

Computer vision:

- facial recognition
- medical imaging
- self-driving cars

Natural Language Processing (NLP):

- translation
- chatbots
- sentiment analysis (assignment)

GenAI:

- images
- music
- text
- videos (deepfakes)

Throwback to 2020

<https://www.youtube.com/watch?v=l82PxsKHxYc>

To create this in 2020, it required:

- 2 hours of professional audio
- 50 hours of HD video
- static pre-defined script
- over \$15K USD of compute

Throwback to 2025

<https://www.youtube.com/watch?v=LDLwShb0Ssg>

Deep learning

neurons that fire together
wire together

Deep learning is a subset of ML algorithms that extract **patterns** from data using neural networks (NN). They are inspired by the structure and function of the human brain.

Deep = NN with many layers



Analogy of layers



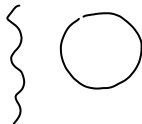
1 Low-level features (contrast, direction)

- edges
- dots
- lines



2 Mid-level features (geometric patterns)

- simple shapes
- circles
- corners
- curves



3 High-level features (combination of mid-level features, spatial relationships)

- two eyes **above** a nose
- smiling or sad face
- face A or face B

Table of Contents

- 1 Deep learning
- 2 Architecture of NN**
- 3 Forward propagation
- 4 Activation function
- 5 Back propagation
- 6 How to structure a NN
- 7 Code example

9:08

Terms overview

Neuron: one single node. Receives input, processes it and passes an output to the next layer.

Weights (w): determine the strength of a connection. The higher the weight, the higher the importance for the final decision.

Biases (b): allowing the model to shift decision-making boundary. The bias is added to each sum.

Layer: stage of the network: input layer, hidden layer, output layer.

Activation function: mathematical function (sigmoid, ReLU) introducing non-linearity. It decides whether a neuron should fire.

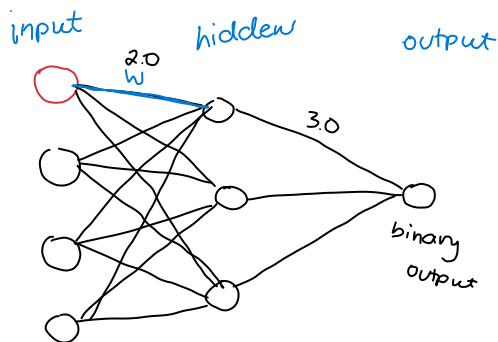
Forward propagation: data travelling from left to right to make a prediction.

Back-propagation: moving right to left to adjust weights and biases, "learning".

Loss function: measuring how far the model's prediction is from the truth. A good model has low loss function.

Single layer NN

- Neuron
- Weights
- Input layer
- Hidden layer
- Output layer



Deep NN

Deep = many hidden layers between the input and the output layer.

Training a NN = finding the best set of weights and biases.

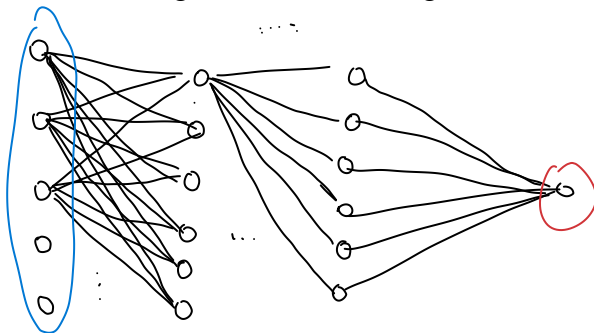


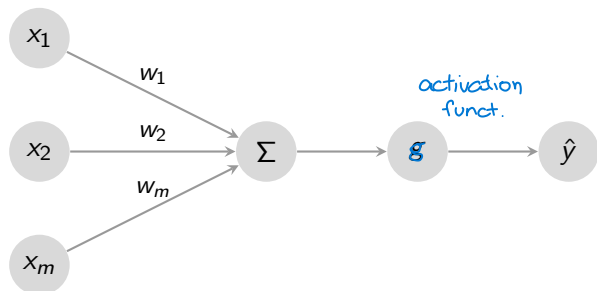
Table of Contents

- 1 Deep learning
- 2 Architecture of NN
- 3 Forward propagation**
- 4 Activation function
- 5 Back propagation
- 6 How to structure a NN
- 7 Code example

Forward propagation: simple example

$$\hat{y} = g \left(\sum_{i=1}^m \underline{x_i w_i} \right)$$

Lin. combinations
of features

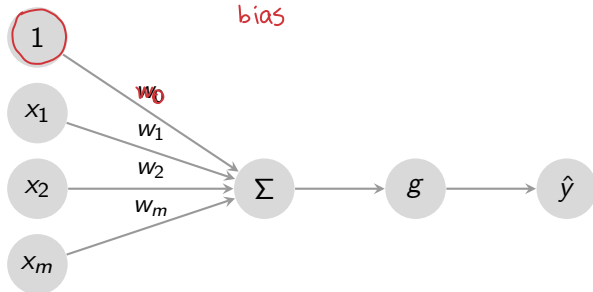


Inputs Weights Sum Non-linearity Output

Forward propagation: introducing bias

$$\hat{y} = g \left(\underline{w_0} + \sum_{i=1}^m x_i w_i \right)$$

bias



Inputs

Weights

Sum

Non-linearity

Output

Forward propagation: vector notation

$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

$$\hat{y} = g \left(w_0 + X^T W \right)$$

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$$

$$W = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix}$$

Forward propagation: example

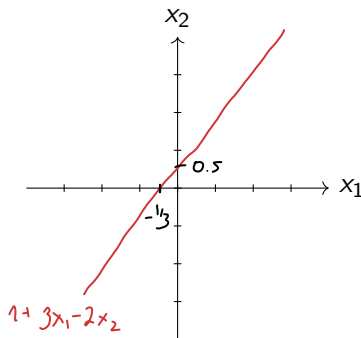
Let $w_0 = 1$ and $W = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$ $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ $g: 27 - 9:34$

$$\hat{y} = g(w_0 + x^T W)$$

$$= g\left(1 + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 \\ -2 \end{bmatrix}\right)$$

$$\hat{y} = g(1 + 3x_1 - 2x_2)$$

a line in 2D
separating into
2 parts



Forward propagation: example

Assume we have input

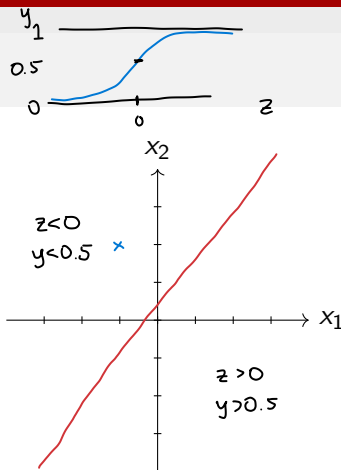
$$\underline{X} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

Use the sigmoid activation function.

$$\hat{y} = g(1 + 3 \cdot (-1) - 2 \cdot 2)$$

$$= g(-6)$$

$$= \frac{1}{1 + e^6} \approx 0.002$$

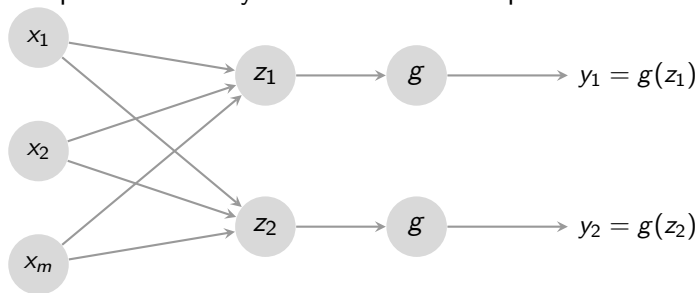


Use sigmoid as your g :

$$g(z) = \frac{1}{1 + e^{-z}}$$

Dense layers

All inputs are densely connected to all outputs = dense.



Inputs

Logits (z) Activation (y)

$$z_i = \underbrace{w_{0,i}}_{\text{bias}} + \sum_{j=1}^m x_j w_{j,i}$$

Table of Contents

- 1 Deep learning
- 2 Architecture of NN
- 3 Forward propagation
- 4 Activation function**
- 5 Back propagation
- 6 How to structure a NN
- 7 Code example

9:45 - 10:05

Activation functions



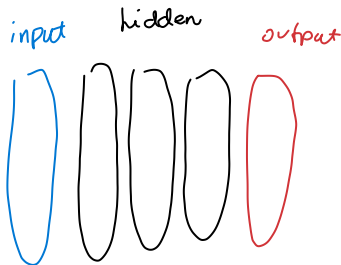
All activation functions are non-linear. Their job is to determine whether a neuron should fire based on its input.

Hidden layer functions:

- ReLU = Rectified Linear Unit
- GELU (used for transformers)
- Leaky ReLU
- Tanh

Output layer functions:

- Softmax *mult. class*
- Sigmoid *bin. class.*
- Linear *reg.*

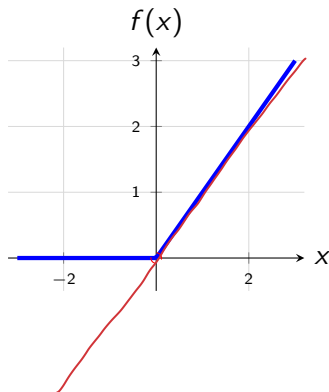


Rectified Linear Unit (ReLU)

The industry standard for hidden layers.

$$f(x) = \max(0, x)$$

- Computationally very **cheap** (no exponents).
- **Sparsity**: Sets inactive neurons to 0.
- Helps mitigate the **Vanishing Gradient** problem.



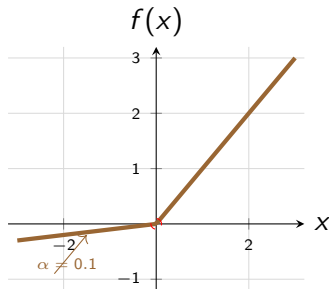
Leaky ReLU

A variation of ReLU that attempts to solve the "Dying ReLU" problem by allowing small negative gradients.

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$

Usually, $\alpha = 0.01$ (slope of 1%).

- **Goal:** Fixes models where too many ReLU neurons permanently output zero.
- Small gradient (α) exists even when $x < 0$.
- **Use Case:** Used when training accuracy stagnates or standard ReLU fails.

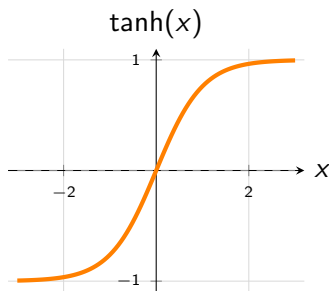


Hyperbolic Tangent (Tanh)

A zero-centered, S-shaped curve.

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- **Range:** strictly $(-1, 1)$.
- **Advantage:** Zero-centered output can make optimization faster than Sigmoid.
- Often used in RNNs.

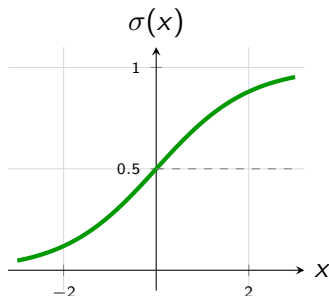


Sigmoid (Logistic) Function

A smooth, S-shaped curve that maps inputs to a probability-like range.

$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

- **Range:** strictly $(0, 1)$.
- Ideal for **Binary Classification** in the output layer.
- Suffers heavily from **Vanishing Gradients** in deep networks.



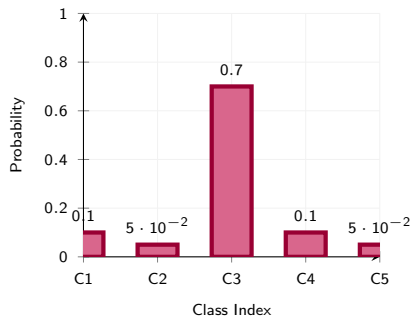
Gradients vanish as $x \rightarrow \pm\infty$ (the curve flattens).

Softmax Function

Converts a vector of raw scores (logits) into a probability distribution that sums to 1.

$$f(x_i) = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}}$$

- **Range:** Outputs strictly in $(0, 1)$.
- **Constraint:** $\sum f(x_i) = 1.0$ (100% total probability).
- Exclusively for **Multi-class Classification** (output layer).



Softmax normalizes outputs to represent class probabilities.

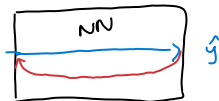
Table of Contents

- 1 Deep learning
- 2 Architecture of NN
- 3 Forward propagation
- 4 Activation function
- 5 Back propagation**
- 6 How to structure a NN
- 7 Code example

Four-step process

- 1 **Forward pass**: input is multiplied with weights, added a bias and passed through an activation function until an output.
- 2 **Loss function**: the prediction is compared to the expected output. Wrong predictions are penalized.
- 3 **Backward pass**: blame the weights that contributed heavily to the wrong answer (high gradient).
- 4 **Weight update**: use an optimizer (SGD/Adam) to fix the blamed weights towards less error.

weights
biases



Loss functions

Regression (MSE)

$$L = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

Binary Cross-Entropy

$$L = -\frac{1}{m} \sum_{i=1}^m (y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i))$$

Multi-class Cross-Entropy

$$L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(\hat{y}_i)$$

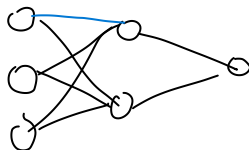
Back-propagation



Changing a weight in the very beginning can have a big impact on the final result (error).

How much does L (loss) change when w changes?

$$\frac{\partial L}{\partial w}$$



Chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

$$z = w \cdot x + b$$

$$a = \sigma(z)$$

↑ act. func.

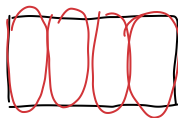
Optimizers

Optimizers determine how the back-propagation is implemented.

- Stochastic Gradient Descent (SGD): most common
random
- RMSProp
- Adaptive Momentum Estimation (Adam)

RMSProp and Adam are both adaptive = automatically adjust the learning rate during training.

Stochastic Gradient Descent



- **Epoch:** one full pass of the entire training set through a NN
- **Batch:** a small subset of the dataset used to calculate a single gradient update
- **Iteration:** completion of one batch

If you have 1000 images and a batch size of 10, then one epoch consists of 100 iterations.

Stochastic Gradient Descent

Stochastic = random

SGD updates weights and biases batch after batch. After going through all samples, it finished a training epoch.

If epochs=10, the SGD optimizer will go through the entire dataset 10 times, updating the weights at every step.

Vanishing and exploding gradients

Gradient = partial derivative $\frac{\partial L}{\partial w}$

$$0.1^{10} = 0.0000000001$$

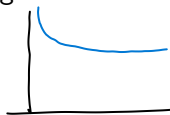
$$100^{10} = 100000000000000000000$$

Vanishing gradient

Very small gradient (between 0 and 1). Multiplying many small numbers shrinks the product to zero.

Caused by sigmoid, tanh (squash large inputs into a very small output range).

The model stops improving even though the error is still high, the training becomes extremely slow.



Exploding gradient

Large gradients (greater than 1). Multiplying many large numbers grows the product to infinity.

Caused by poor weight initialization or deep networks (eg. RNN).

Weights change drastically in a single step. The loss function

might return NaN and model's
accuracy suddenly crash to 0.

Table of Contents

- 1 Deep learning
- 2 Architecture of NN
- 3 Forward propagation
- 4 Activation function
- 5 Back propagation
- 6 How to structure a NN**
- 7 Code example

11:00

Input and layer

^
output

The size of your input layer should match the size of your input.

8x8 image: 64

Dataframe with 7 columns: 7

The size of your output layer should match your output.

Binary classification: 1

Multiclass classification: as many as classes

Hidden layers

Number of hidden layers:

- Simple problems: 1 hidden layer is often sufficient
- Moderately complex problems: 2-3 hidden layers work well for many real-world applications
- Complex problems (complex image recognition): consider a deep network with 4+ layers

Number of neurons per layer:

- Rule of 2/3: number of neurons in hidden layer = $\frac{2}{3}$ size of input layer + size of output layer
- Rule of thumb: hidden layer size should be between the size of input and output layers
- Pyramid shape: start with more neurons and gradually decrease (128, 64, 32)

Architecture tip

Start with a **small network** and add complexity only when needed. Sometimes complexity only adds computations time without bringing any benefits.

Regularization

To prevent overfitting:

- L1 and L2
- Dropout
- Early stopping

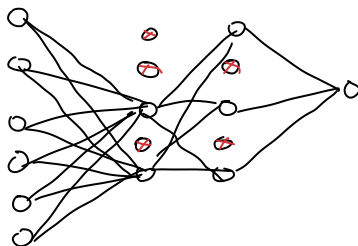
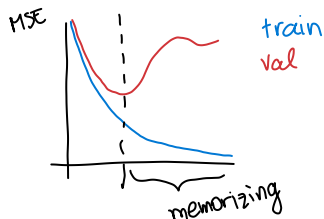


Table of Contents

- 1 Deep learning
- 2 Architecture of NN
- 3 Forward propagation
- 4 Activation function
- 5 Back propagation
- 6 How to structure a NN
- 7 Code example**

Code

Github link: https://github.com/304125/MAL1_S26/tree/master/09_Neural_Networks